

[Home](#) / [Blog](#) / Terragrunt Tutorial:...

Terragrunt Tutorial: Examples and Use Cases



Sam Gabrail

President, TeKanAid

Hi there! How can we help?



What is Terragrunt?

Terragrunt is a thin wrapper for Terraform that provides extra tools for keeping your configurations DRY (Don't Repeat Yourself). With Terragrunt, you can easily manage multiple environments. It also helps you keep your codebase clean and organized.

Why use Terragrunt?

There are several reasons to use Terragrunt over just using pure Terraform code. Be and we will elaborate more under the Terragrunt Features section.

1. DRY code and configurations
2. Versioning and environment management

3. Dependency management
4. Hooks for custom actions
5. Keep your remote state configuration DRY
6. Keep your CLI flags DRY
7. Execute Terraform commands on multiple modules at once
8. Work with multiple AWS accounts

Getting Started with Terragrunt

Video Walk-through



What is Terragrunt

env0

28:12

Requirements: A GitHub account (all the hands-on sections will utilize GitHub's CoC won't need to install anything on your machine)

TL;DR: You can find [the repo here](#).

Installing Terragrunt

To install Terragrunt, download the binary for your operating system from the [Releases](#) page and add it to your PATH. Alternatively, you could use a [package manager as shown here](#). If you work with us with codespaces, you will have all the binaries already installed for you.

Basic Terragrunt Commands

Instead of running Terraform commands directly, you run the same commands with

- terragrunt init -> equivalent to [terraform init](#)
- terragrunt plan -> equivalent to [terraform plan](#)
- terragrunt apply -> equivalent to [terraform apply](#)
- terragrunt output -> equivalent to terraform output
- terragrunt destroy -> equivalent to [terraform destroy](#)

These commands will call the corresponding Terraform commands, with Terragrunt additional logic before and after the Terraform calls.

Using Terragrunt for Infrastructure Manag

Terragrunt can be used to manage your infrastructure configurations, plans, and [Terraform](#). You can define your Terragrunt configuration in a **terragrunt.hcl file**, which allows you to manage versions of your Terraform modules and fill in variables specific to each environment. We'll see an example later.

Terragrunt Features

Let's now elaborate more on the key features that Terragrunt offers:

1. **DRY code:**

Terragrunt helps you avoid duplicating code by allowing you to define common environment-specific variables.

2. **Infrastructure management:**

Terragrunt simplifies the management of multiple environments by providing a way to switch between them.

3. **Versioning:**

Terragrunt allows you to reference specific versions of your Terraform module in your environment, making it easier to manage and roll back changes.

4. **Hooks:**

Terragrunt supports hooks that can be used to perform actions before or after running Terraform commands.

5. **Keep your remote state configuration DRY:**

Terragrunt allows you to define your backend block once in a root **terragrunt.hcl** file and reuse it in all your Terraform environments. Terragrunt can also automatically create the necessary locking resources (such as S3 buckets and DynamoDB tables) for you.

6. **Keep your CLI flags DRY:**

You can configure Terragrunt to pass specific CLI arguments for specific commands in your **terragrunt.hcl** file.

7. **Execute Terraform commands on multiple modules at once:**

Instead of manually running `terraform apply` in each of the subfolders corresponding to different environments and waiting for them to complete, you can use Terragrunt with `terragrunt apply` to deploy multiple Terraform modules at once.

8. Work with multiple AWS accounts:

TerraGrunt allows you to work with multiple AWS accounts by letting you specify an IAM role to assume for each account. You can use the `--terragrunt-iam-role` command line option to specify the **TERRAGRUNT_IAM_ROLE** environment variable to tell TerraGrunt which role to assume. TerraGrunt then calls the **sts assume-role** API on your behalf and exposes the credentials in environment variables when running Terraform. This way, you can manage your infrastructure across different accounts without having to store your AWS credentials in plaintext or without having to manually call assume-role every time, and without having to modify your Terraform code or backend configuration.

TerraGrunt Workflow and Best Practices

To make the most of TerraGrunt, follow these best practices:

- **Structuring Configurations:** Organize your Terraform code into modules and use version constraints to reference these modules with specific versions.
- **Managing Dependencies:** Use TerraGrunt to manage dependencies between your Terraform components.
- **Using TerraGrunt Hooks:** Utilize hooks to perform actions before or after Terraform runs.
- **Automation with CI/CD:** Integrate TerraGrunt with your CI/CD pipeline for automated deployment.

TerraGrunt Benefits and Drawbacks

While TerraGrunt offers many benefits, it also has some drawbacks, such as:

- It adds an additional layer of complexity to your infrastructure management during the initial setup.
- It is also another tool to manage

- Doesn't work with Terraform Cloud

However, if you are using env0, you can make full use of Terragrunt's benefits because [the few tools that support Terragrunt](#).

Yevgeniy Brikman, who is the co-founder of Gruntworks (the company that brought you Terraform Cloud), makes a [great comparison](#) between using Terraform workspaces, Git branches, and Terraform Cloud, and summed up his comparison with the table below:

	Workspaces	Branches	Terraform Cloud
Minimize code duplication	■ ■ ■ ■ ■	□ □ □ □ □	■
See and navigate environments	□ □ □ □ □	■ ■ ■ □ □	■
Different settings in each environment	■ ■ ■ ■ ■	■ ■ ■ ■ □	■
Different backends for each environment	□ □ □ □ □	■ ■ ■ ■ □	■
Easy to manage multiple backends	□ □ □ □ □	■ ■ ■ ■ □	■
Different versions in each environment	□ □ □ □ □	■ ■ □ □ □	■
Share data between modules	■ ■ □ □ □	■ ■ □ □ □	■
Work with multiple modules concurrently	□ □ □ □ □	□ □ □ □ □	■
No extra tooling to learn or use	■ ■ ■ ■ ■	■ ■ ■ ■ ■	□
Works with Terraform Cloud	■ ■ ■ ■ ■	■ ■ ■ ■ ■	■

Terragrunt Use Cases and Examples

Terragrunt can be used in various scenarios, such as managing infrastructure for different development, staging, and production. In our example, we will see how to use Terraform configuration. We will first run everything with pure Terraform only then to improve our configuration using Terragrunt.

Furthermore, to keep things simple, we will only consider two environments: dev and

Our WordPress Module

We will use an example WordPress application to showcase the difference between Terraform only and using Terragrunt. This example was taken from [this repo](#), but modified to our needs. The WordPress Terraform module creates the following resources:

- A VPC
- 1 Public Subnet for the EC2 instance
- 2 Private Subnets for the RDS
- An Internet Gateway
- A route table
- Security groups for EC2 and RDS
- EC2 instance for the WordPress application
- RDS instance for the MySQL database for WordPress

You can check the Terraform code in our repo.

Terraform Only Scenario

Below is the folder structure when using Terraform only.


```
├─ environments
│   ├── dev
│   │   ├── main.tf
│   │   └─ outputs.tf
│   └─ prod
│       ├── main.tf
│       └─ outputs.tf
└─ modules
    └─ wordpress
        ├── aws_ami.tf
        ├── main_script.tf
        ├── outputs.tf
        ├── user_data.tpl
        ├── userdata_ubuntu.tpl
        ├── variables.tf
        └─ versions.tf
```

Under each environment folder, you can see that we're duplicating code in both the **outputs.tf** files. Even though we are using a terraform module structure, we still duplicate unnecessarily. Recall that using terraform modules is a great way for code reuse.

Below is the content of the **main.tf** file for the dev environment.

```
terraform {
  backend "s3" {
    bucket      = "tekanaid-terragrunt-demo"
    key         = "wordpress/dev/terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
    dynamodb_table = "my-lock-table"
```

```

    }
  }

  module aws_wordpress {
    source                = "../../modules/wordpress"
    database_name         = "wordpress_db"    // database name
    database_user         = "wordpress_user"  //database username
    database_password     = "dev-PassWord4-user" //password for user
    region                = "us-east-1"
    IsUbuntu              = true
    AZ1                   = "us-east-1a" // for EC2
    AZ2                   = "us-east-1b" //for RDS
    AZ3                   = "us-east-1c" //for RDS
    VPC_cidr              = "10.0.0.0/16"      // VPC CIDR
    subnet1_cidr          = "10.0.1.0/24"      // Public Subnet for EC2
    subnet2_cidr          = "10.0.2.0/24"      //Private Subnet for RDS
    subnet3_cidr          = "10.0.3.0/24"      //Private subnet for RDS
    PUBLIC_KEY_PATH       = "./mykey-pair.pub"
    PRIV_KEY_PATH         = "./mykey-pair"
    instance_type         = "t2.micro"         //type of instance
    instance_class        = "db.t2.micro"      //type of RDS Instance
    root_volume_size     = 22
  }

```

Below is the content of the **main.tf** file for the prod environment.

```

terraform {
  backend "s3" {
    bucket      = "tekanaid-terragrunt-demo"
    key         = "wordpress/prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
  }
}

```

```

    dynamodb_table = "my-lock-table"
  }
}

module aws_wordpress {
  source              = "../modules/wordpress"
  database_name       = "wordpress_db"    // database name
  database_user       = "wordpress_user"  //database username
  database_password   = "prod-PassWord4-user" //password for user
  region              = "us-east-2"
  IsUbuntu            = true
  AZ1                 = "us-east-2a" // for EC2
  AZ2                 = "us-east-2b" //for RDS
  AZ3                 = "us-east-2c" //for RDS
  VPC_cidr            = "10.10.0.0/16"    // VPC CIDR
  subnet1_cidr        = "10.10.1.0/24"    // Public Subnet for EC2
  subnet2_cidr        = "10.10.2.0/24"    //Private Subnet for RDS
  subnet3_cidr        = "10.10.3.0/24"    //Private subnet for RDS
  PUBLIC_KEY_PATH     = "./mykey-pair.pub"
  PRIV_KEY_PATH       = "./mykey-pair"
  instance_type       = "t2.small"       //type of instance
  instance_class      = "db.t2.small"    //type of RDS Instance
  root_volume_size    = 22
}

```

There are two main factors to notice between the two **main.tf** files:

1. The backend configuration is different and it is quite error-prone to copy and paste configuration. You may accidentally use the prod state file in dev and vice versa and override an existing state file. Variables are not allowed to be used in the backend block.
2. Some of the input variables to the **aws_wordpress** module are duplicated. In that case, it is not a big deal, however, when you have multiple terraform modules you will

difference.

Now let's take a look at the output variables in the **outputs.tf** file for the dev environ

```
output "IP" {  
  value = module.aws_wordpress.IP  
}  
  
output "RDS-Endpoint" {  
  value = module.aws_wordpress.RDS-Endpoint  
}  
  
output "INFO" {  
  value = module.aws_wordpress.INFO  
}
```

Notice that they are exactly the same. So again this is violating the DRY principle and when you have many other environments and applications.

Deploy with Terraform only

Follow the instructions below to deploy the WordPress application using pure Terra

In the **Terraform_Only/environments/dev** folder create a private/public key pair with passphrase:

```
ssh-keygen -f mykey-pair  
sudo chmod 400 mykey-pair
```

In order to create a remote backend to store Terraform state files, you will need to create an S3 bucket and a Dynamo DB table in AWS. [This is a good guide.](#)

Run these Terraform commands:

```
terraform init
terraform plan
terraform apply
```

To deploy the production Wordpress application, run the same steps above but in the **Terraform_Only/environments/prod** folder.

Here is the output of the **terraform apply** command:

```
Apply complete! Resources: 15 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
INFO = "AWS Resources and Wordpress has been provisioned. Go to http://54.167.129.51"
IP = "54.167.129.51"
RDS-Endpoint = "terraform-20230526182044784600000004.crxircqkii6t.us-east-1.rds.amazonaws.com"
```

And going to the 'http://54.167.129.51' address shows you the WordPress setup screen.



English (United States)

Afrikaans

አማርኛ

Aragonés

العربية

العربية المغربية

অসমীয়া

گۆنئی آذربایجان

Azərbaycan dili

Беларуская мова

Български

বাংলা

བོད་ཡིག

Bosanski

Català

Cebuano

Čeština

Cymraeg

Dansk

Deutsch (Schweiz, Du)

Deutsch (Österreich)

Deutsch (Sie)

Continue

Terragrunt Scenario

Now let's take a look at how Terragrunt can improve our Terraform code structure.

Below is the folder and file structure in our local file system, when using Terragrunt. a root **terragrunt.hcl** configuration file and a **terragrunt.hcl** configuration file per environment.

```
├── environments
│   ├── dev
│   │   └── terragrunt.hcl
│   ├── prod
│   │   └── terragrunt.hcl
│   └── terragrunt.hcl
└── modules
    └── wordpress
        ├── aws_ami.tf
        ├── main_script.tf
        ├── outputs.tf
        ├── user_data.tpl
        ├── userdata_ubuntu.tpl
        ├── variables.tf
        └── versions.tf
```

The Root Terragrunt Configuration File

Now let's take a look at the main or root Terragrunt configuration file called **terragrunt.hcl** in the **environments** folder.

```
remote_state {
  backend = "s3"
  config = {
    bucket = "tekanaid-terragrunt-demo"
```

```
key          =  
"terragrunt/wordpress/${path_relative_to_include()}/terraform.tfstate"  
region       = "us-east-1"  
encrypt      = true  
dynamodb_table = "my-lock-table"  
}  
  
inputs = {  
  ami_id          = "ami-0c55b159cbfafa1f0" # Amazon Linux 2 AMI  
  database_name    = "wordpress_db"          // database name  
  database_user    = "wordpress_user"        //database username  
  IsUbuntu         = true  
  PUBLIC_KEY_PATH  = "./mykey-pair.pub"  
  PRIV_KEY_PATH    = "./mykey-pair"  
  root_volume_size = 22  
}
```

Notice how the `key` in the **remote_state** block is parameterized. Each environment has its own key without us worrying about copying and pasting. So the dev environment's key is

```
terragrunt/wordpress/dev/terraform.tfstate
```

Whereas the prod environment's key will be:

```
terragrunt/wordpress/prod/terraform.tfstate
```


The second thing to notice is that we define the input variables that are common to environments here. Once again this shows how we are following the DRY principle.

Now let's take a look at the dev TerraGrunt configuration files under both the **environments/prod** folders.

Below is the **terragrunt.hcl** file under the **environments/dev** folder.

```
include {
  path = find_in_parent_folders()
}

terraform {
  source = "../../modules//wordpress"
}

inputs = {
  database_password = "dev-PassWord4-user" //password for user da
  region            = "us-east-1"
  AZ1               = "us-east-1a"        // for EC2
  AZ2               = "us-east-1b"        //for RDS
  AZ3               = "us-east-1c"        //for RDS
  VPC_cidr          = "10.0.0.0/16"       // VPC CIDR
  subnet1_cidr      = "10.0.1.0/24"       // Public Subnet for EC2
  subnet2_cidr      = "10.0.2.0/24"       //Private Subnet for RDS
  subnet3_cidr      = "10.0.3.0/24"       //Private subnet for RDS
  instance_type     = "t2.micro"          //type of instance
  instance_class     = "db.t2.micro"      //type of RDS Instance
}
```

and below is the **terragrunt.hcl** under the **environments/prod** folder.

```

include {
  path = find_in_parent_folders()
}

terraform {
  source = "../../../modules//wordpress"
}

inputs = {
  database_password = "prod-PassWord4-user" //password for user d
  region            = "us-east-2"
  AZ1                = "us-east-2a"         // for EC2
  AZ2                = "us-east-2b"         //for RDS
  AZ3                = "us-east-2c"         //for RDS
  VPC_cidr           = "10.10.0.0/16"       // VPC CIDR
  subnet1_cidr       = "10.10.1.0/24"       // Public Subnet for EC2
  subnet2_cidr       = "10.10.2.0/24"       //Private Subnet for RDS
  subnet3_cidr       = "10.10.3.0/24"       //Private subnet for RDS
  instance_type      = "t2.small"          //type of instance
  instance_class     = "db.t2.small"       //type of RDS Instance
}

```

In both the dev and prod TerraGrunt configuration files you can see that we're including the root TerraGrunt config file using the `find_in_parent_folders()` function.

We're also including the input variables that are specific to each environment. This is

The second thing to notice is the source attribute inside the **terraform** block. Here we're including the Terraform module **Wordpress** that exists two levels above inside the **modules** folder. It's possible to reference a terraform module that lives in a git repository, which is actually a common pattern. In this case, another benefit of using TerraGrunt is that you can source different Terraform modules for different environments. For example, the dev environment

version v0.0.2 of our **Wordpress** module whereas the prod environment is still on v0.0.1. Since the dev environment has been properly tested you can then upgrade the prod environment to v0.0.2. With pure Terraform, you can't parameterize the **source** block rendering all environments to use with the same version of the module unless you hard-code the version values.

Deploy with Terragrunt

Follow the instructions below to deploy the WordPress application using pure Terraform.

In the **Terragrunt/environments/dev** folder create a private/public key pair with an ssh key.

```
ssh-keygen -f mykey-pair  
sudo chmod 400 mykey-pair
```

Then run the following Terragrunt commands:

```
terragrunt init  
terragrunt plan  
terragrunt apply
```

Terragrunt sets the AWS remote backend for you. It will create an S3 bucket and a local cache directory.

To deploy the production WordPress application, run the same steps above but in the **Terragrunt/environments/prod** folder.

Here is the output of the `terragrunt apply` command:

```
Apply complete! Resources: 15 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

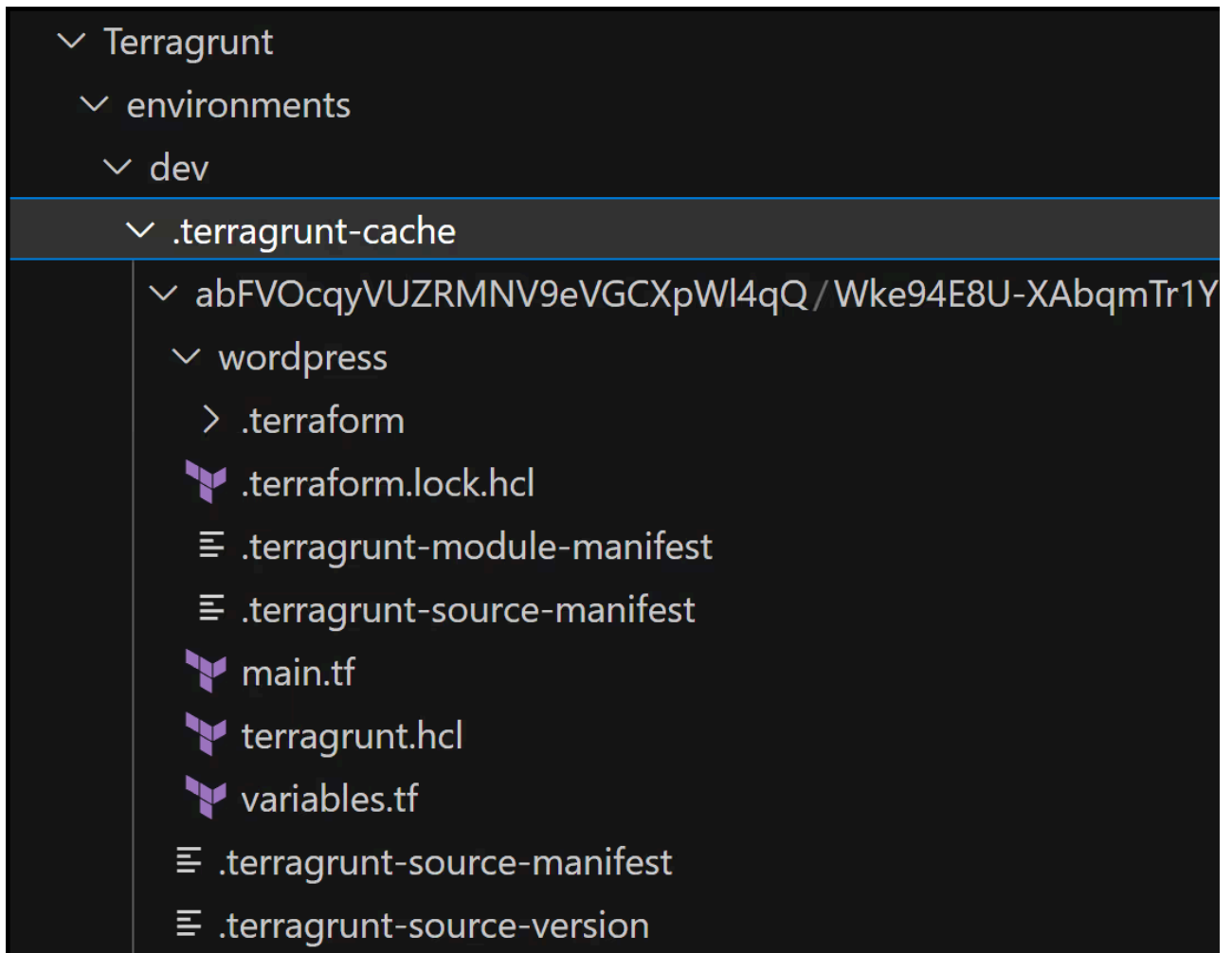
```
INFO = "AWS Resources and Wordpress has been provisioned. Go to http://18.213.69.210"
IP = "18.213.69.210"
RDS-Endpoint = "terraform-20230526183958929700000004.crxircqkii6t.us-east-1.rds.amazonaws.com"
```

Notice the Terragrunt output is exactly the same as the previous Terraform only screenshot. We didn't need to define an **outputs.tf** file to output the variables from the module. Terragrunt does this for us for free.

And going to the URL shows you the same WordPress setup screen that we saw earlier.

Terragrunt Cache

The Terragrunt cache is a folder that Terragrunt creates in the current working directory. It contains downloaded Terraform configurations, modules, providers, and backend settings. Terragrunt uses this cache to avoid downloading the same code multiple times and to speed up the execution of Terraform commands. You can safely delete this folder at any time and Terragrunt will recreate it. You can also change the location of this folder by setting the `TERRAGRUNT_DOWNLOAD` variable. Here is what it looks like



Cleanup and the run-all command

TerraGrunt has a neat feature that allows you to run commands across multiple fold will run the following command to destroy the dev and prod environments in paralle **TerraGrunt/environments** folder:

```
terragrunt run-all destroy
```

Below is the output showing both the dev and prod environments destroyed succe:

```
Destroy complete! Resources: 15 destroyed.
aws_db_instance.wordpressdb: Still destroying... [id=te
0526200051689800000004, 4m20s elapsed]
aws_db_instance.wordpressdb: Destruction complete after
aws_security_group.RDS_allow_rule: Destroying... [id=sg
3781de]
aws_db_subnet_group.RDS_subnet_grp: Destroying... [id=t
305262000474972000000002]
aws_db_subnet_group.RDS_subnet_grp: Destruction complet
aws_subnet.prod-subnet-private-2: Destroying... [id=sub
25f25b45f]
aws_subnet.prod-subnet-private-1: Destroying... [id=sub
fec2acf84]
aws_subnet.prod-subnet-private-1: Destruction complete
aws_subnet.prod-subnet-private-2: Destruction complete
aws_security_group.RDS_allow_rule: Destruction complete
aws_security_group.ec2_allow_rule: Destroying... [id=sg
685a8d]
aws_security_group.ec2_allow_rule: Destruction complete
aws_vpc.prod-vpc: Destroying... [id=vpc-0ef12d67b2c3f0a
aws_vpc.prod-vpc: Destruction complete after 0s
Releasing state lock. This may take a few moments...

Destroy complete! Resources: 15 destroyed.
```

Terragrunt with env0

As we've seen, Terragrunt offers several benefits over Terraform and helps to address challenges inherent to Terraform implementations. Since Terragrunt is built on top of Terraform, it benefits from env0 management.

When you use env0 in conjunction with Terragrunt, you get the following benefits:

- Automate your Terragrunt deployments in CI pipelines for Infrastructure as Code
- Manage your Terragrunt variables centrally and securely.

- Integrate your Terragrunt-based environments with other environments management tools.
- Combine multiple Terragrunt deployments to create more sophisticated environments.
- Use [Terragrunt run-all](#) to execute commands on multiple modules efficiently.
- Manage your underlying Terraform state safely and easily.
- Increase the reusability and extensibility of your Terragrunt deployments.

Key Takeaways

Terragrunt is a powerful tool that enhances the Terraform experience by providing features like remote state, workspaces, versioning, dependency management, and hooks, among other features. By following best practices and leveraging Terragrunt's features, you can create a more efficient and maintainable infrastructure management process in large-scale deployments. Combine env0 with Terragrunt to unlock the full potential of your Infrastructure as Code strategy.

References

- <https://terragrunt.gruntwork.io/docs/getting-started/quick-start/>
- <https://blog.gruntwork.io/terragrunt-how-to-keep-your-terraform-code-dry-and-reusable-61ae06959d8>
- <https://jhooq.com/terragrunt-guide/>
- <https://github.com/orgs/gruntwork-io/discussions/92>
- <https://blog.devops.dev/a-complete-overview-of-terragrunt-fbebb53fbd42>
- <https://itnext.io/structuring-terraform-project-using-terragrunt-part-i-4c6e936c>
- <https://github.com/gruntwork-io/terragrunt-infrastructure-live-example>

Related Content



BLOG

Expanding AI in env0: PR and Error Summaries



Hadas Weinrib

June 5
11, • min
2025 read



BLOG

Expanding Drift Remediation: Keep Your Code Aligned with Cloud Changes



Hadas Weinrib

May 5
7, • min
2025 read



BLOG

Drift Cause Analysis: Context That Drives Confident Action

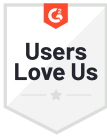


Yuval Nelinger

April 5
7, • min
2025 read

Schedule a **technical demo**. See env0 in action.

Get a Demo

	Available on:	Company	Developer and DevOps	Alt
Founding member: 	 AWS Marketplace	About Us	Documentation	Terra
	 Azure Marketplace	Case Studies	API	Cloud
 	 GDC Marketplace	Careers	env0 Terraform Provider	Alter
		Newsroom	Terratag Open Source	Terra
		Contact Us		Cloud
				Atlas
				Alter
				env0 only

© Copyright
env0 2025

Terms of
Service

Privacy
Policy

Security

System
Status

Search

